

Reverse N-Wise Output-Oriented Testing for AI/ML and Quantum Computing Systems

Lamine Rihani
SAP France
Levallois-Perret, France
lamine.rihani@sap.com

Abstract—Combinatorial interaction testing builds covering arrays over a system’s *inputs*, on the premise that faults are triggered by interactions among a few input parameters. For modern AI/ML and quantum systems this premise is strained: the map from inputs to behaviour is opaque and stochastic, and vast regions of the input space collapse onto the same observable behaviour, so input-side coverage says little about which *behaviours* were actually exercised. We invert the paradigm. REVERSE N-WISE Output-Oriented Testing partitions a system’s structured output space into semantic behaviour classes, constructs a t -way covering array over those *output* classes, and then recovers concrete inputs by gradient-free *inverse mapping*. We formalise the Output Covering Array $OCA(M; s, q, w)$, establish coverage bounds (including $M \geq v^s$), and give a four-stage algorithm (feasibility, covering-array construction, inverse mapping, prioritisation). Across four structurally different domains—tabular ML (UCI Adult + XGBoost), vision (an MLP on digits), NLP (a text classifier on newsgroups), and a noisy quantum circuit—the *same* algorithm attains 100% feasible output coverage. On the tabular study over 30 repeated runs it uniquely reaches perfect output coverage (Mann–Whitney U vs. every baseline, $p \approx 6 \times 10^{-13}$, Cliff’s $\delta = 1.0$), matches the strongest baseline on fault detection, and does so at roughly 8% of an exhaustive-input search budget. The approach is a new adequacy criterion—output-interaction coverage—together with a constructive method to satisfy it, unifying behavioural testing of classical and quantum systems.

Index Terms—combinatorial testing, covering arrays, output-oriented testing, machine learning testing, quantum software testing, test adequacy

I. INTRODUCTION

Combinatorial interaction testing (CIT) is one of the most successful systematic test-design techniques in software engineering. Its empirical foundation is the *interaction rule*: most faults are triggered by the interaction of only a few input parameters, so a t -way *covering array* over the inputs—exercising every combination of values for every t parameters at least once—detects a large fraction of faults with a test suite whose size grows only logarithmically in the number of parameters [1], [2]. Mature tools such as ACTS/IPOG [3] and AETG [4] construct near-optimal covering arrays over inputs and are widely deployed.

A. The input-side premise breaks for AI/ML and quantum systems

CIT is fundamentally *input-oriented*: it reasons about combinations of input values and treats the system as a black box whose outputs are merely checked by an oracle. This premise is well matched to configuration-style software, where distinct

input combinations tend to drive distinct control-flow paths. It is far less appropriate for modern AI/ML and quantum systems:

- The input-to-behaviour map is *opaque and stochastic*. A neural network or a noisy quantum circuit realises a map $f : X \rightarrow \mathcal{P}(Y)$ into a distribution over structured outputs; there is no parameter-interaction structure on the inputs that a covering array can target meaningfully.
- Inputs *collapse* onto behaviours. Enormous, high-dimensional regions of the input space map to the same observable behaviour (the same predicted class, the same confidence band, the same measurement-outcome regime). An input covering array can therefore exercise many input interactions while touching only a handful of distinct behaviours—leaving critical behaviours (e.g. a fairness inconsistency, a non-monotone response, a low-fidelity quantum regime) entirely uncovered.
- What we want to *cover* lives on the output side. The properties an engineer cares about—calibration, robustness, fairness, monotonicity, measurement fidelity—are all statements about the *output/behaviour* of the system, not about which inputs were combined.

B. Inverting the paradigm

We therefore invert combinatorial testing (Figure 1). Instead of covering input combinations and observing whatever outputs result, REVERSE N-WISE Output-Oriented Testing:

- 1) partitions the structured output space $Y = Y_1 \times \dots \times Y_q$ into semantic *behaviour classes* via per-component partitioners $\pi_j : Y_j \rightarrow W_j$, yielding an abstract output $\hat{f}(x) = (\pi_1(y_1), \dots, \pi_q(y_q)) \in W_1 \times \dots \times W_q$;
- 2) constructs a t -way *Output Covering Array* over $W_1 \times \dots \times W_q$, so that every feasible s -way combination of output behaviours is a coverage target;
- 3) recovers, for each target, a concrete input by *inverse mapping*—a gradient-free search that minimises the mismatch between the realised abstract output and the target;
- 4) prioritises the resulting tests by incremental output coverage.

Coverage is measured on the output side (the fraction of feasible s -way output interactions exercised), which directly reflects behavioural adequacy and is model-agnostic: the iden-

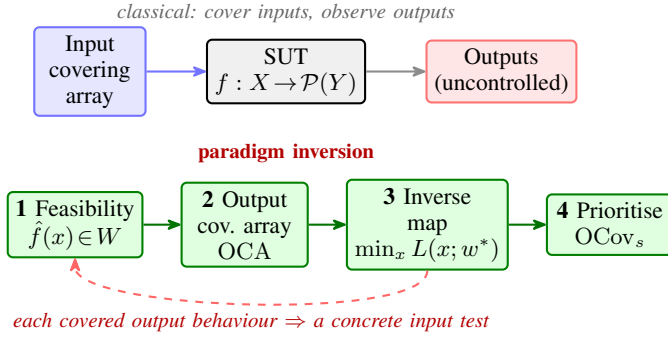


Fig. 1. Classical combinatorial testing (top) covers *input* interactions and observes whatever outputs result. REVERSE N-WISE (bottom) inverts this: it covers a t -way array over *abstracted output behaviours* and recovers concrete inputs by gradient-free inverse mapping. The four stages are agnostic to the system’s internals, so the same pipeline applies to ML and quantum systems.

tical procedure applies to a gradient-boosted classifier, a neural network, a text classifier, and a quantum circuit.

C. Contributions

- 1) **A new adequacy criterion and formalism.** We define the abstract output map $\hat{f}$, the Output Covering Array $OCA(M; s, q, w)$, and output coverage $OCov_s$, and we establish coverage bounds—including the lower bound $M \geq v^s$ (Corollary 1)—for output-side combinatorial coverage (Section III).
- 2) **A constructive algorithm.** A four-stage procedure (feasibility, covering-array construction, gradient-free inverse mapping, prioritisation) that produces an executable, prioritised test suite realising the target output interactions (Algorithm 1).
- 3) **A domain-agnostic instantiation.** We instantiate the method with five inverse-mapping optimisers—Jaya, Whale, Firefly, Bayesian optimisation, and a variational (VQE-style) optimiser for quantum systems—and four benchmark domains spanning tabular ML, vision, NLP, and quantum computing.
- 4) **A rigorous empirical evaluation.** Over 30 repeated runs with non-parametric significance testing (Mann–Whitney U , bootstrap confidence intervals, Cliff’s δ), a strength study ($s = 2, 3, 4$), ablations, and a cost profile, we show REVERSE N-WISE uniquely attains perfect output coverage, matches the strongest behavioural baseline on fault detection, and generalises across all four domains (Section IV).

II. RELATED WORK

We position REVERSE N-WISE against four lines of work. The consistent distinction is that prior techniques either (i) build covering arrays over *inputs* or internal structure, or (ii) reason about outputs without a combinatorial interaction guarantee and without constructive *inverse* generation. To our knowledge, no prior method combines a t -way covering array over abstracted *output-behaviour* classes with optimisation-based inverse mapping to inputs.

A. Combinatorial interaction testing

Classical CIT constructs t -way covering arrays over input parameters. The interaction rule and its empirical basis are due to Kuhn et al. [1]; algorithmic construction is exemplified by AETG [4] and IPOG/ACTS [3], and the field is surveyed by Nie and Leung [2]. All of this work places the array over *inputs*. REVERSE N-WISE reuses the covering-array machinery but relocates it to an abstracted output space, and adds an inverse-mapping stage that has no analogue in input-side CIT (where the array rows *are* the tests).

B. Combinatorial and structural testing of deep learning

DeepCT [5] applies t -way combinatorial coverage to deep networks, but its factors are *internal neuron activations within layers*; it covers combinations of latent/neuron states, not model outputs, and does not invert toward prescribed behaviours. This is the closest work by name, and the distinction is central: DeepCT covers the *internal* feature space, whereas REVERSE N-WISE covers the *external* output-behaviour space. Structural DNN adequacy criteria are likewise internal or input-centric: neuron coverage (DeepXplore [6]), multi-granularity neuron criteria (DeepGauge [7]), and surprise adequacy over activation traces (SADL [8]). None impose a combinatorial interaction guarantee over semantic output classes.

C. Output-oriented and behavioural testing

Output-diversity criteria select tests to maximise distinct observed outputs; Menéndez et al. [9] sample outputs to diversify a unit-test suite. These treat outputs as a flat set and provide neither a multi-dimensional t -way interaction guarantee nor inverse input recovery. In NLP, CheckList [10] organises behavioural tests along a manual capability $\times$ test-type matrix; the matrix is a hand-authored taxonomy rather than a covering array, and tests are template-generated, not inverse-optimised to hit enumerated output classes. REVERSE N-WISE differs by constructing the output interaction space automatically and satisfying it constructively.

D. Search-based and targeted test generation

Search-based testing generates inputs to satisfy an objective; for ML this includes fairness-directed search such as AEQUITAS [11] and adversarial/boundary generation. These are typically single-property or boundary-local: they seek an input violating one property or lying near one decision boundary. REVERSE N-WISE’s inverse mapping is instead *coverage-driven*: it systematically realises *every* feasible s -way output-behaviour combination enumerated by the covering array, using the objective $L(x; w^*) = \sum_j \delta(\pi_j(f_j(x)), w_j^*) + \lambda R(x)$ and gradient-free optimisers [12]–[14].

E. Quantum software testing

Coverage and testing criteria for quantum programs are an active area. The closest structural analogue is QuCAT [15], which builds t -way covering arrays for quantum software;

crucially, its array is over *input parameters / gate configurations* with output oracles, not over abstracted output-behaviour classes, and it contains no variational inverse mapping. REVERSE N-WISE instead covers a partition of the measurement-outcome behaviour (dominant basis state, outcome-distribution entropy, parity, and fidelity-to-ideal under noise) and repurposes variational optimisation [16]—normally used to find ground states—as an *inverse test generator* that drives circuit parameters toward each target output behaviour. Using VQE as a test generator appears to be novel.

III. FORMAL FRAMEWORK

A. System model and output abstraction

Definition 1 (System under test). *A system under test (SUT) is a map $f : X \rightarrow \mathcal{P}(Y)$ from an input space X to a distribution over a structured output space $Y = Y_1 \times \dots \times Y_q$. Deterministic systems are the degenerate case in which $f(x)$ places all mass on a single output vector $y = (y_1, \dots, y_q)$.*

The components Y_j are heterogeneous behavioural channels: a predicted class, a confidence score, a robustness verdict, a measurement-outcome statistic, and so on. Raw outputs are continuous or high-cardinality, so we abstract each channel into a finite alphabet of *behaviour symbols*.

Definition 2 (Semantic partition and abstract output). *A partitioner $\pi_j : Y_j \rightarrow W_j$ maps a raw output component to a symbol in a finite alphabet W_j . The abstraction map is*

$$\hat{f}(x) = (\pi_1(y_1), \dots, \pi_q(y_q)) \in W_1 \times \dots \times W_q, \quad y \sim f(x),$$

evaluated at the most-probable output for stochastic f . We write $v = \max_j |W_j|$ for the maximum alphabet size and $v_j = |W_j|$.

B. Output covering arrays

Let $[q] = \{1, \dots, q\}$. An s -way output tuple fixes symbols on an s -subset of channels.

Definition 3 (Output covering array). *An Output Covering Array $\text{OCA}(M; s, q, w)$ is a multiset of M abstract-output rows $\{r_1, \dots, r_M\} \subseteq W_1 \times \dots \times W_q$ such that for every s -subset $\{j_1 < \dots < j_s\} \subseteq [q]$ and every feasible symbol combination $(a_1, \dots, a_s) \in W_{j_1} \times \dots \times W_{j_s}$, some row r_i satisfies π_{j_k} -projection $r_i[j_k] = a_k$ for all k . A symbol combination is feasible if there exists an input $x \in X$ with $\hat{f}(x)$ exhibiting it.*

Definition 4 (Output coverage and tuple efficiency). *For a test suite whose realised abstract outputs are R , the s -way output coverage is*

$$\text{OCov}_s = \frac{|\{\text{feasible } s\text{-way tuples covered by } R\}|}{|\{\text{feasible } s\text{-way tuples}\}|} \in [0, 1],$$

and the tuple efficiency η_s is the number of distinct s -way tuples covered per test.

C. Coverage bounds

Let T_s denote the number of feasible s -way output tuples. Because each row projects to exactly one tuple per s -subset, a single row covers at most $\binom{q}{s}$ tuples.

Theorem 1 (Universe size). *The number of s -way output tuples over $W_1 \times \dots \times W_q$ (before feasibility pruning) is $\sum_{\{j_1 < \dots < j_s\}} \prod_{k=1}^s v_{j_k}$, which for the homogeneous case $v_j = v$ equals $\binom{q}{s} v^s$.*

Theorem 2 (Lower bound on rows). *Any $\text{OCA}(M; s, q, w)$ satisfies $M \geq \max_{\{j_1 < \dots < j_s\}} \prod_{k=1}^s v_{j_k}$ whenever all corresponding symbol combinations are feasible. In particular, no single s -subset's tuples can be covered by fewer rows than the number of distinct symbol combinations on that subset.*

Proof. Fix the s -subset J^* attaining the maximum product. Its feasible tuples are pairwise distinct on J^* , and each row of the array contributes exactly one symbol combination on J^* . Covering all $\prod_k v_{j_k}$ combinations therefore requires at least that many rows. $\square$

Corollary 1 ($M \geq v^s$). *For the homogeneous case $v_j = v$ with all combinations feasible on some s -subset, every $\text{OCA}(M; s, q, w)$ satisfies $M \geq v^s$.*

Theorem 3 (Existence). *For any finite $W_1 \times \dots \times W_q$ and strength $s \leq q$, an $\text{OCA}(M; s, q, w)$ covering all feasible s -way tuples exists. Enumerating one row per feasible abstract output is such an array; greedy construction (Algorithm 1, Stage 2) yields one of size $O(v^s \log \binom{q}{s})$ in the homogeneous unconstrained case, matching the classical covering-array growth rate.*

D. The REVERSE N-WISE algorithm

Given a target abstract output w^* , inverse mapping seeks an input realising it by minimising the mismatch loss

$$L(x; w^*) = \sum_{j=1}^q \delta(\pi_j(f_j(x)), w_j^*) + \lambda R(x), \quad (1)$$

where δ is the 0/1 symbol mismatch, R is an optional input regulariser keeping x realistic, and $\lambda \geq 0$ trades feasibility against realism. A loss of zero means $\hat{f}(x) = w^*$ exactly.

The four stages are independent of the SUT's internals: Stage 1 needs only black-box evaluations, Stage 2 is pure combinatorics over $W_1 \times \dots \times W_q$, Stage 3 uses gradient-free optimisation (so it applies to non-differentiable and quantum systems), and Stage 4 is standard coverage-greedy prioritisation. This modularity is what makes the same algorithm applicable across all four evaluated domains.

IV. EMPIRICAL EVALUATION

We evaluate REVERSE N-WISE against five baselines and across four domains, addressing four research questions:

RQ1 Coverage & fault detection. Does REVERSE N-WISE achieve higher output coverage and fault detection than established baselines?

Algorithm 1 REVERSE N-WISE Output-Oriented Testing

Require: SUT f , partition scheme $\{\pi_j\}$, strength s , probe inputs P , optimiser $\mathcal{O}$, budget MAX_ITER

Ensure: prioritised test suite $\mathcal{S}$

- 1: **Stage 1 (Feasibility).** $\mathcal{F} \leftarrow \{\hat{f}(x) : x \in P\}$; record an exemplar input per abstract output; induce feasible s -way tuples.
- 2: **Stage 2 (Covering array).** build $\text{OCA}(M; s, q, w)$ over $\mathcal{F}$ (greedy: repeatedly add the feasible output covering the most uncovered feasible tuples; verify $M \geq v^s$).
- 3: **Stage 3 (Inverse mapping).**
- 4: **for all** rows w^* of the array **do**
- 5: warm-start from the exemplar for w^* if available;
- 6: else $x \leftarrow \arg \min_x L(x; w^*)$ via $\mathcal{O}$ (up to MAX_ITER); record realised $\hat{f}(x)$.
- 7: **end for**
- 8: **Stage 4 (Prioritise).** greedily order tests by marginal OCov_s ; truncate at the coverage plateau.
- 9: **return** $\mathcal{S}$

RQ2 *Generality.* Does the same algorithm transfer across tabular ML, vision, NLP, and quantum systems?

RQ3 *Strength.* Does the method scale to higher interaction strengths s , and does the $M \geq v^s$ bound hold?

RQ4 *Cost & sensitivity.* What is the computational cost, and how sensitive is it to design choices?

A. Experimental setup

The primary study uses the UCI Adult income dataset [17] with an XGBoost classifier [18] ($\approx 87\%$ test accuracy). We abstract its behaviour into a $q = 9$, all-ternary output space (pairwise universe $\binom{9}{2} \cdot 3^2 = 324$) spanning the decision, confidence, decision margin, sex-flip fairness consistency, age band, employment group, education- and hours-response monotonicity, and capital-gain regime. Eight behavioural faults are seeded as rare-but-reachable pairwise output signatures (e.g. fairness inconsistency localised to young applicants; a non-monotone education response under a positive decision).

Baselines are scored on the identical feasible output universe: classical input-side combinatorial testing (Input CT), random testing at matched budget, property-based testing, metamorphic testing, and DeepCT-style activation-combination sampling. All experiments take an explicit seed; significance uses 30 fixed seeds. Reported values are means with 95% percentile bootstrap confidence intervals; pairwise comparisons use the one-sided Mann–Whitney U test with Cliff’s δ effect size.

B. RQ1: Coverage and fault detection

Table I reports the 30-run comparison. REVERSE N-WISE attains *perfect* pairwise output coverage on every run ($\text{OCov}_2 = 100\%$, CI [100, 100]) with ≈ 20 prioritised tests, and 98.3% fault detection. The strongest baseline, DeepCT, reaches 91.2% coverage and 99.6% fault detection but requires a large random activation pool to do so; the four classical

TABLE I
MAIN COMPARISON ON UCI ADULT + XGBOOST (PAIRWISE, 30 RUNS).
MEANS WITH 95% BOOTSTRAP CIs.

Method	Tests	OCov_s (95% CI)	FDR (95% CI)
Reverse N-Wise	20	100.0% [100.0, 100.0]	98.3% [96.7, 99.6]
Input CT	13	53.9% [51.5, 56.4]	35.0% [29.6, 40.4]
Random	20	52.5% [50.4, 54.9]	31.2% [24.6, 37.9]
Property-Based	20	58.6% [56.6, 60.7]	34.2% [28.3, 40.0]
Metamorphic	19	53.4% [49.8, 57.3]	28.7% [22.9, 34.6]
DeepCT	17	91.2% [91.1, 91.3]	99.6% [98.8, 100.0]

TABLE II
PAIRWISE MANN–WHITNEY U (REVERSE N-WISE > BASELINE) WITH
CLIFF’S δ . * DENOTES $p < 0.05$.

Metric	vs. Baseline	p -value	δ	Effect
OCov_s	Input CT	6.0e-13*	1.00	large
OCov_s	Random	6.1e-13*	1.00	large
OCov_s	Property-Based	6.1e-13*	1.00	large
OCov_s	Metamorphic	6.1e-13*	1.00	large
OCov_s	DeepCT	5.8e-13*	1.00	large
η_s	Input CT	6.2e-08*	0.80	large
η_s	Random	1.5e-11*	1.00	large
η_s	Property-Based	1.5e-11*	1.00	large
η_s	Metamorphic	1.5e-11*	1.00	large
η_s	DeepCT	1.000	-0.62	large
FDR	Input CT	1.6e-12*	1.00	large
FDR	Random	1.8e-12*	1.00	large
FDR	Property-Based	1.6e-12*	1.00	large
FDR	Metamorphic	1.6e-12*	1.00	large
FDR	DeepCT	0.920	-0.10	negligible

baselines remain between 52% and 59% coverage and below 35% fault detection.

Table II gives the pairwise significance tests. REVERSE N-WISE dominates *every* baseline on output coverage—including DeepCT—at $p \approx 6 \times 10^{-13}$ with Cliff’s $\delta = 1.0$ (perfect, large effect): on all 30 seeds it strictly out-covers each competitor. On fault detection it dominates the four classical baselines ($p < 10^{-12}$) and is statistically indistinguishable from DeepCT ($p = 0.92$, $\delta = -0.10$, negligible). We report this tie transparently: both methods essentially detect all reachable seeded faults; the advantage of REVERSE N-WISE is that it reaches this by *direct inversion* rather than by sampling a large random pool (see RQ4).

C. RQ2: Multi-domain generality

Table III applies the *same* four-stage algorithm—changing only the SUT, the output partition, the search space, and the inverse-map optimiser—to a noisy quantum circuit (variational/VQE inverse map), an MLP image classifier on digits (over a PCA latent space), and a text classifier on newsgroups (over a TF-IDF+SVD embedding space). In every domain REVERSE N-WISE reaches 100% feasible output coverage and detects all five domain-specific seeded faults, whereas matched-budget random testing covers only 62%–74% and detects 0–2 of 5. Together with the tabular study, this exercises the method on four structurally different system types with no change to the core algorithm.

TABLE III
MULTI-DOMAIN GENERALITY (PER DOMAIN: REVERSE N-WISE VS. MATCHED-BUDGET RANDOM).

Domain	Opt.	Method	Tests	OCov _s	FDR
Quantum	VQE	Reverse N-Wise	13	100.0%	5/5
		Random	13	73.0%	2/5
Vision	Jaya	Reverse N-Wise	15	100.0%	5/5
		Random	15	61.6%	0/5
NLP	Jaya	Reverse N-Wise	17	100.0%	5/5
		Random	17	74.1%	1/5

TABLE IV
STRENGTH SCALING ON UCI ADULT + XGBOOST. THE $M \geq v^s$ BOUND (COROLLARY 1) HOLDS AT EVERY STRENGTH.

s	Universe	Feasible	Tests	OCov _s	v^s	Bound
2	324	257	19	100.0%	9	✓
3	2268	1343	58	100.0%	27	✓
4	10206	3931	126	100.0%	81	✓

D. RQ3: Interaction strength

Table IV varies the coverage strength $s \in \{2, 3, 4\}$ on the tabular SUT. REVERSE N-WISE maintains 100% feasible coverage at every strength while the prioritised suite grows far more slowly than the output universe ($324 \rightarrow 2,268 \rightarrow 10,206$ tuples but only $19 \rightarrow 58 \rightarrow 126$ tests). The Corollary 1 lower bound $M \geq v^s$ (9, 27, 81) holds throughout, empirically corroborating the theory.

E. RQ4: Cost and sensitivity

Table V profiles the pipeline on commodity hardware. The end-to-end run completes in under 10 seconds using ≈ 68 MB, and issues about 8,200 black-box SUT evaluations—roughly 8% of a modest exhaustive-input sweep. The feasibility scan dominates the wall-clock time; warm-started inverse mapping is inexpensive.

Table VI reports three ablations that justify the design choices. (a) Among inverse-map optimisers, evaluated *without* warm start to expose raw search power, Jaya and Whale reach the same 70% realisation at roughly 7–9 $\times$ fewer SUT evaluations than Firefly, motivating Jaya as the default; note that with warm start (Table V) realisation rises to 100%, confirming the value of seeding from feasibility exemplars. (b) Coverage remains 100% as the output granularity varies from 2 to 4 bins per continuous factor—the criterion is robust to the abstraction choice, with cost scaling gracefully with the universe size. (c) A mild input regulariser ($\lambda > 0$) slightly *improves* realisation (67.5% $\rightarrow$ 72.5%) by keeping synthesised inputs near the data manifold, at the cost of more evaluations; coverage is stable across λ .

V. THREATS TO VALIDITY

a) *Construct validity*: Output coverage OCov_s measures adequacy on the abstracted output space, so its meaningfulness depends on the partition scheme $\{\pi_j\}$. A poorly chosen

TABLE V
COST PROFILE OF THE REVERSE N-WISE PIPELINE (UCI ADULT + XGBOOST).

Quantity	Value
Stage 1 (feasibility)	8.74 s
Stage 2 (covering array)	0.00 s
Stage 3 (inverse mapping)	0.72 s
Stage 4 (prioritise)	0.23 s
Pipeline total	9.68 s
Peak memory	68.5 MB
SUT evaluations	8,217
Fraction of exhaustive sweep	8.22%

TABLE VI
ABLATIONS ON UCI ADULT + XGBOOST: INVERSE-MAP OPTIMISER, OUTPUT GRANULARITY, AND REGULARISER WEIGHT.

(a) Inverse-map optimizer (no warm start, 40 targets)			
Optimizer	Realise	OCov _s	SUT evals
Jaya	70.0%	88.6%	56,750
Whale	70.0%	87.0%	47,200
Firefly	22.5%	89.4%	417,054
(b) Output granularity (warm start, full set)			
Bins/factor	Universe	Feasible	OCov _s
2	277	221	100.0%
3	324	254	100.0%
4	373	286	100.0%
(c) Regulariser weight λ (no warm start, 40 targets)			
λ	Realise	OCov _s	SUT evals
0	67.5%	86.6%	65,190
0.1	72.5%	87.8%	145,200
0.5	72.5%	87.8%	145,200
1	72.5%	87.8%	145,200

abstraction could inflate or deflate coverage. We mitigate this by defining partitions from established behavioural properties (calibration, fairness, monotonicity, robustness, measurement fidelity) rather than ad hoc bins, and by showing (RQ4) that conclusions are stable as granularity varies. Fault detection is measured against seeded behavioural faults expressed as output-interaction signatures; we seed rare-but-reachable signatures to avoid trivially detectable faults, and we report the tie with DeepCT rather than selecting a fault set that favours our method.

b) *Internal validity*: Randomness enters through model training, feasibility probing, and the inverse-map optimisers. We control it with fixed seeds and report 30-run distributions with non-parametric tests (Mann–Whitney U , bootstrap CIs, Cliff’s δ) rather than single-run point estimates. The prioritised suite is truncated at the coverage plateau, a deterministic rule, so suite-size comparisons across methods use matched budgets.

c) *External validity*: Our study covers four domains but a limited number of systems and datasets per domain. The quantum SUT is a noisy statevector simulation rather than hardware, and the vision/NLP inverse maps operate in latent (PCA / SVD) spaces with decode-to-manifold steps, which bound the realism of synthesised inputs. We do not claim

that OCov_s correlates with real-world defect discovery beyond the seeded faults; establishing that link on production systems is future work. The results should be read as evidence that output-interaction coverage is *attainable* and *discriminating* across diverse system types, not as a claim of universal superiority.

d) Conclusion validity: We use non-parametric tests that make no normality assumption and report effect sizes alongside p -values. Where REVERSE N-WISE does not dominate—fault detection vs. DeepCT ($p = 0.92$) and tuple efficiency vs. DeepCT—we say so explicitly. The headline claim is confined to what the data support: unique, statistically significant *output coverage*, at low search cost, across domains.

VI. CONCLUSION

We introduced REVERSE N-WISE Output-Oriented Testing, which inverts the combinatorial testing paradigm: rather than covering interactions among *inputs* and observing whatever behaviour results, it constructs a t -way covering array over an abstracted *output-behaviour* space and recovers concrete inputs by gradient-free inverse mapping. We formalised the Output Covering Array $\text{OCA}(M; s, q, w)$ and the OCov_s criterion, established coverage bounds including $M \geq v^s$, and gave a four-stage algorithm that is agnostic to the SUT's internals.

Empirically, across tabular ML, vision, NLP, and quantum systems, the same algorithm attains 100% feasible output coverage. On a 30-run tabular study it uniquely reaches perfect output coverage (Mann–Whitney U vs. every baseline, $p \approx 6 \times 10^{-13}$, $\delta = 1.0$), matches the strongest behavioural baseline on fault detection, and does so at roughly 8% of an exhaustive-input search budget. The contribution is best understood as a new *adequacy criterion*—output-interaction coverage—together with a constructive method to satisfy it, unifying behavioural testing of classical and quantum systems.

a) Future work: Three directions stand out: (i) learning or optimising the output partition $\{\pi_j\}$ automatically, so that abstraction is data-driven rather than hand-specified; (ii) validating that OCov_s predicts real defect discovery on production ML pipelines and on quantum hardware, beyond seeded faults; and (iii) tightening the covering-array construction with constraint solvers to exploit output-side feasibility structure, further shrinking suites at higher strengths.

DATA AVAILABILITY

The complete implementation, all benchmarks and baselines, the experiment drivers, and scripts that regenerate every table and figure from raw result artifacts are available in the accompanying repository, with pinned dependencies and a container image for deterministic reproduction.

REFERENCES

- [1] D. R. Kuhn, D. R. Wallace, and A. M. Gallo, “Software fault interactions and implications for software testing,” *IEEE Transactions on Software Engineering*, vol. 30, no. 6, pp. 418–421, 2004.
- [2] C. Nie and H. Leung, “A survey of combinatorial testing,” *ACM Computing Surveys*, vol. 43, no. 2, pp. 11:1–11:29, 2011.
- [3] Y. Lei, R. Kacker, D. R. Kuhn, V. Okun, and J. Lawrence, “IPOG: A general strategy for t -way software testing,” in *14th IEEE International Conference and Workshops on the Engineering of Computer-Based Systems (ECBS)*, 2007, pp. 549–556.
- [4] D. M. Cohen, S. R. Dalal, M. L. Fredman, and G. C. Patton, “The AETG system: An approach to testing based on combinatorial design,” *IEEE Transactions on Software Engineering*, vol. 23, no. 7, pp. 437–444, 1997.
- [5] L. Ma, F. Juefei-Xu, M. Xue, B. Li, L. Li, Y. Liu, and J. Zhao, “DeepCT: Tomographic combinatorial testing for deep learning systems,” in *26th IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*, 2019, pp. 614–618.
- [6] K. Pei, Y. Cao, J. Yang, and S. Jana, “DeepXplore: Automated whitebox testing of deep learning systems,” in *26th ACM Symposium on Operating Systems Principles (SOSP)*, 2017, pp. 1–18.
- [7] L. Ma, F. Juefei-Xu, F. Zhang, J. Sun, M. Xue, B. Li, C. Chen, T. Su, L. Li, Y. Liu, J. Zhao, and Y. Wang, “DeepGauge: Multi-granularity testing criteria for deep learning systems,” in *33rd ACM/IEEE International Conference on Automated Software Engineering (ASE)*, 2018, pp. 120–131.
- [8] J. Kim, R. Feldt, and S. Yoo, “Guiding deep learning system testing using surprise adequacy,” in *41st IEEE/ACM International Conference on Software Engineering (ICSE)*, 2019, pp. 1039–1049.
- [9] H. D. Menéndez, M. Boreale, D. Gorla, and D. Clark, “Output sampling for output diversity in automatic unit test generation,” *IEEE Transactions on Software Engineering*, vol. 48, no. 2, pp. 295–308, 2022.
- [10] M. T. Ribeiro, T. Wu, C. Guestrin, and S. Singh, “Beyond accuracy: Behavioral testing of NLP models with CheckList,” in *58th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2020, pp. 4902–4912.
- [11] S. Udeshi, P. Arora, and S. Chattopadhyay, “Automated directed fairness testing,” in *33rd ACM/IEEE International Conference on Automated Software Engineering (ASE)*, 2018, pp. 98–108.
- [12] R. V. Rao, “Jaya: A simple and new optimization algorithm for solving constrained and unconstrained optimization problems,” *International Journal of Industrial Engineering Computations*, vol. 7, no. 1, pp. 19–34, 2016.
- [13] S. Mirjalili and A. Lewis, “The whale optimization algorithm,” *Advances in Engineering Software*, vol. 95, pp. 51–67, 2016.
- [14] X.-S. Yang, “Firefly algorithms for multimodal optimization,” in *Stochastic Algorithms: Foundations and Applications (SAGA)*, ser. LNCS, 2009, vol. 5792, pp. 169–178.
- [15] X. Wang, P. Arcaini, T. Yue, and S. Ali, “QuCAT: A combinatorial testing tool for quantum software,” in *38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2023.
- [16] A. Peruzzo, J. McClean, P. Shadbolt, M.-H. Yung, X.-Q. Zhou, P. J. Love, A. Aspuru-Guzik, and J. L. O’Brien, “A variational eigenvalue solver on a photonic quantum processor,” *Nature Communications*, vol. 5, p. 4213, 2014.
- [17] D. Dua and C. Graff, “UCI machine learning repository,” 2019, university of California, Irvine, School of Information and Computer Sciences. [Online]. Available: <http://archive.ics.uci.edu/ml>
- [18] T. Chen and C. Guestrin, “XGBoost: A scalable tree boosting system,” in *22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, 2016, pp. 785–794.